Operating System Unit 2: Process and Threads

1) Process Concept

- **Process = Program in execution**

- A process is **active** → has:

1. Program code
2. Input/Output
3. **State** (values of variables, CPU registers, etc.)

- **Parallelism** means doing **more than one task at the *same time***.

True Parallelism

Pseudo Parallelism

2) PROCESS CREATION

Processes are created when the system needs to run a task.
Four main events lead to process creation:

Event	Meaning / Example
System initialization	When OS starts (booting).
User request to run a program	Opening Chrome, Word, etc.
Execution of a process creation system call	Parent process creates child.
Batch job initiation	On mainframes (grouped tasks executed later).

Process Creation in UNIX

- UNIX uses **`fork()`** system call.
- It **clones** the calling process → creates **child process**.
- Then child typically calls **`execve()`** to replace its memory with a *new program*.

Simplified:

```
Parent (fork) → Child (execve → runs new program)
```

Process Creation in Windows

- Uses **`CreateProcess()`**.
- This *creates the* ***process*** *AND* ***loads the program*** in **one step**.

3) PROCESS TERMINATION

A process ends with **`exit()`** system call (Unix) or `ExitProcess` (Windows).

Reasons for termination:

Type	Meaning/Example
Normal exit (Voluntary)	Completed successfully.
Error exit (Voluntary)	Program error (e.g., file not found).
Fatal Error (Involuntary)	Illegal memory access, divide by zero.
Killed by another process (Involuntary)	`kill` command in Unix, `TerminateProcess` in Windows.

4) PROCESS STATES

A process moves through different **states** during execution.

State	Meaning
New	Being created
Ready	Waiting for CPU
Running	Currently using CPU
Waiting (Blocked)	Waiting for I/O or event
Terminated	Finished execution

5) PROCESS STATE TRANSITIONS

Transition No.	Name	From → To	Meaning
1	**Admitted**	New → Ready	Process created & moved into ready queue
2	**Dispatch**	Ready → Running	CPU scheduler selects process
3	**Block**	Running → Blocked	Process waits for I/O or event
4	**Time-Out**	Running → Ready	CPU time slice finished
5	**Wake-Up**	Blocked → Ready	Event completed (e.g., I/O done)
6	**Exit**	Running → Terminated	Process finishes or is killed

6) Process Management

1) Information maintained by OS for managing processes

The OS keeps certain information to track and control each process. The most important pieces:

Term	Meaning
Process Context	The current state of a process required to resume it later.
Process Control Block (PCB)	A data structure in the OS that stores all information about a process.
CPU Virtualization	OS makes each process believe it has its own CPU.
Context Switching	Saving current process state and loading another.
Dispatching Loop	The cycle where the OS scheduler selects the next process to run.

7) Process Context

Definition:

A **Process Context** is **all runtime information** of a process required to **resume it after being paused**.

It includes:

- **User-level context**:
  - Code (instructions)
  - Data (global/static variables)
  - Stack (function calls + local variables)
  - Heap (dynamic memory, e.g., `new`, `malloc`)

8) Process Control Block (PCB)

A **PCB** stores essential information about a process **throughout its lifetime**.

Stored in PCB:

- Process ID (PID)
- Process state (running / ready / waiting / terminated)
- CPU Register values
- Program Counter
- Memory management info (page tables, segment info)
- I/O status and file descriptors

The OS keeps **one PCB per process** in a **Process Table**.

9) Cooperating Processes

Types:

Type	Effect on Others?	Example
Independent	No	Calculator app separate from music player
Cooperating	Yes	Browser + Download Manager share data

Why cooperation is useful?

- **Information sharing**
- **Faster execution** (divide work into parallel tasks)
- **Modularity** (clean design)
- **Convenience** (edit + print + compile at same time)

10) Inter-Process Communication (IPC)

Two main mechanisms:

IPC Method	Pros	Cons	Usage
Message Passing	Simple, no shared memory issues	Slower (kernel involvement)	Sending small data between processes
Shared Memory	Very fast (direct memory access)	Needs synchronization to avoid conflicts	Large data exchange (databases, browsers)

11) Threads

A **thread** = a path of execution within a process (a "lightweight process").

Each thread has:

- Its own **Program Counter**
- Its own **Registers**
- Its own **Stack**

But threads **share**:

- Code
- Data section
- Heap
- OS resources (files, sockets)

Why use threads?

- They allow **parallel** execution *inside the same process*.

- Common in servers (e.g., one thread per client connection).

Thread stack

Each thread gets its **own stack**, because each thread has its own function call flow.

12) Process vs Thread

Similarities (Slide 28)

Threads behave like processes in some ways:

Similarity	Meaning
Both share the CPU	Only **one** thread/process runs *at a time* on one core.
Both execute sequentially	Each follows step-by-step execution.
Both can create children	`fork()` for processes, thread libraries for threads.
If one is blocked, another can run	Scheduler switches to another runnable task.

Differences (Slide 29)

Process	Thread
Independent	**Not independent** (belong to the same process)
Have **separate** address spaces	**Share** the same address space
Do **not necessarily assist** each other	**Designed to assist** one another
Heavyweight	Lightweight

✔ **Key idea:**
All threads inside a process see the same memory.
So one thread can read/modify variables used by another thread.

13) Thread Model (Slides 30–32)

A **process** consists of:

- **Address space** (code + data + heap + stack memory pool)
- **One or more threads**

A **thread** consists of:

- **Program Counter (PC)**
- **Registers**
- **Own stack**

Threads vs Processes (in memory)

Property	Processes	Threads
Address space	Separate	Shared
Stack	Separate	Each thread has own stack
Registers / PC	Separate	Separate
Resource sharing	Limited	Very high

 Because memory is shared, **threads can overwrite each other's data**, making **synchronization important**.

⍰ Advantages of Threads over Multiple Processes (Slides 33–34)

1) Faster Context Switching

- Switching between threads is faster than between processes.
- Because threads share memory → **less state to save/restore**.

2) Resource Sharing

- Threads share:
  - Code section

    - Data section
    - Open files
- This allows **easy communication**.

⚠ Disadvantages of Threads (Slides 35–36)

1) Blocking

If the kernel is **single-threaded**:

- A blocking system call in one thread **blocks the whole process**.

2) Security Risks

- Since memory is shared, one bug/corrupted thread can:
    - Modify another thread's data
    - Overwrite another thread's stack
    - Crash entire process

14) Thread Model (Slides 30–32)

A **process** consists of:

- **Address space** (code + data + heap + stack memory pool)
- **One or more threads**

A **thread** consists of:

- **Program Counter (PC)**
- **Registers**
- **Own stack**

Threads vs Processes (in memory)

Property	Processes	Threads
Address space	Separate	Shared
Stack	Separate	Each thread has own stack
Registers / PC	Separate	Separate

Property	Processes	Threads
Resource sharing	Limited	Very high

 Because memory is shared, **threads can overwrite each other's data**, making **synchronization important**.

Advantages of Threads over Multiple Processes (Slides 33–34)

1) Faster Context Switching

- Switching between threads is faster than between processes.
- Because threads share memory → **less state to save/restore**.

2) Resource Sharing

- Threads share:
  - Code section
  - Data section
  - Open files
- This allows **easy communication**.

Disadvantages of Threads (Slides 35–36)

1) Blocking

If the kernel is **single-threaded**:

- A blocking system call in one thread **blocks the whole process**.

2) Security Risks

- Since memory is shared, one bug/corrupted thread can:
  - Modify another thread's data
  - Overwrite another thread's stack
  - Crash entire process

Thread States (Slide 37)

Same as process states:

State	Meaning
Ready	Waiting to be assigned CPU
Running	Currently executing
Blocked	Waiting for I/O or event
Terminated	Finished execution

Thread Usage (Slides 38–39)

Threads are useful when:

1. **Multiple tasks inside a program occur simultaneously.**
   Example: MS Word → typing + spell check + autosave.
2. **They are easy to create and destroy**
   - Threads are created **100× faster** than processes.
3. **Performance gain**
   - Useful if tasks involve **I/O + computation**, not only CPU.
4. **Multiprocessor systems**
   - Threads can run **truly in parallel** on multiple CPUs.

Thread Implementation Methods (Slide 40)

There are **three** ways threads can be implemented:

1. **User-Level Threads (ULT)**
2. **Kernel-Level Threads (KLT)**
3. **Hybrid Threads (ULT + KLT)**

---

1) User-Level Threads (ULT) (Slides 41–43)

Threads are managed **in user space**, NOT by the OS.

- Implemented by **thread libraries** (e.g., POSIX threads, Java threads)
- The **OS does not know** threads exist.
- The OS sees **one process**, even if it has many threads.
- Thread switching happens **without system calls** → no context switch to kernel → **very fast**.

Advantages of User-Level Threads

Advantage	Meaning
No OS modification needed	Works even on OS that don't support threads
Simple representation	Each thread has PC, registers, stack in user memory
Fast context switching	Switching threads is like calling a function → very fast
Custom scheduling	Application can decide which thread to run

Disadvantages of User-Level Threads

Disadvantage	Meaning
No kernel awareness	OS treats process as a single unit
No true parallelism	On multi-core, still only one thread runs at a time
Blocking problem	If **one thread blocks**, the **whole process blocks**
Need non-blocking system calls	Example: must avoid normal blocking I/O calls

---

2) Kernel-Level Threads (KLT) (Slides 44–46)

Here, the **OS kernel itself manages threads**.

- Kernel maintains a **Thread Control Block (TCB)** for each thread.
- Scheduler selects **threads**, not processes.
- Thread operations require **system calls** → involve kernel → **slower than ULT**.

Advantages of Kernel-Level Threads

Advantage	Meaning
True parallelism	Multiple threads can run on **multiple CPUs**

Advantage	Meaning
No blocking issue	If one thread blocks, others continue
Good for I/O-heavy programs	Scheduler can switch automatically

Disadvantages of Kernel-Level Threads

Disadvantage	Meaning
Slower	System calls needed for thread operations
Heavy weight	Needs kernel data structures (TCB) → more memory & complexity

---

3) Hybrid Threads (Slide 47)

This is a **combination** of User-Level & Kernel-Level threads.

- Some threads are managed in **user space**
- Some scheduling decisions are **delegated to the kernel**
- Allows **many ULTs to be mapped to fewer KLTs**
  (called **M:N Model** → M user threads mapped to N kernel threads)

Goal

 Keep **speed** of ULT
 Keep **true parallelism** of KLT

⍰ Super-Short Exam Summary

Feature	User-Level Threads	Kernel-Level Threads	Hybrid Threads
Managed by	Thread Library	OS Kernel	Both
Speed	**Fast**	**Slow**	Medium
Parallelism on multi-core	No	Yes	Yes
Blocking issue	Yes	No	No

Feature	User-Level Threads	Kernel-Level Threads	Hybrid Threads
Complexity	Low	High	Medium